

Verilog vs SystemVerilog:

• **Verilog** is a hardware description language (HDL) used to model, simulate, and design digital systems.

• **SystemVerilog** is an extension of Verilog that adds features for both design and verification.

• Initially developed as a verification language, it was later adopted as a standard HDL for both design and verification.

• It includes all Verilog capabilities but also adds significant enhancements, including object-oriented programming (OOP) features and advanced verification methodologies.

RTL

REGISTER TRANSFER LEVEL

At this level, you describe how data moves between registers and how logic transforms that data

- Digital hardware is all about:
- ➔ • **Inputs** (signals coming in)
- ➔ • **Logic** (the combinational part doing calculations)
- ➔ • **State** (the sequential part storing values)

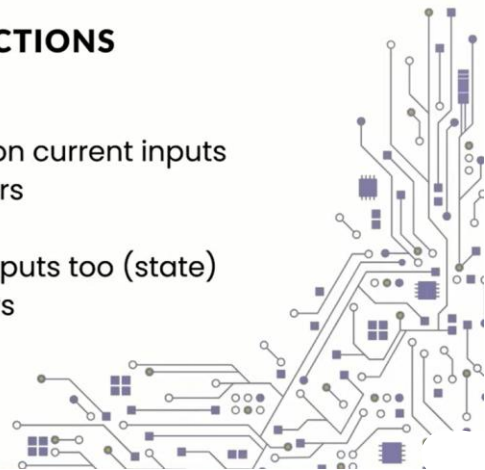


Logic Elements

PERFORM LOGIC FUNCTIONS

Three basic types of logic elements:

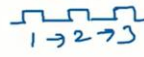
- ➔ 1. **Combinational Logic** – Output depends only on current inputs
 - Examples: AND, OR, NOT, multiplexers, adders
 - No memory, no clock
- ➔ 2. **Sequential Logic** – Output depends on past inputs too (state)
 - Examples: Flip-flops, counters, shift registers
 - Needs a clock signal to work
- ➔ 3. **Mixed Logic** – Most designs combine both

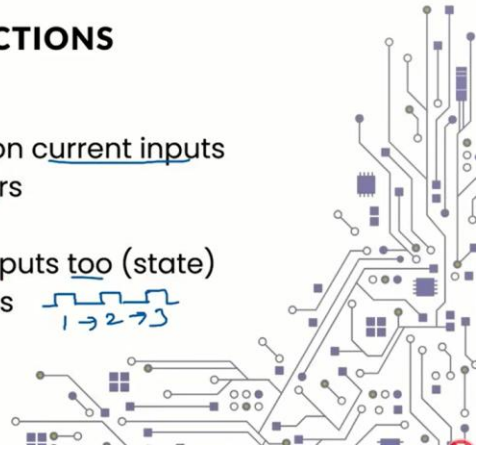


Logic Elements

PERFORM LOGIC FUNCTIONS

Three basic types of logic elements:

- 1. **Combinational Logic** – Output depends only on current inputs
 - Examples: AND, OR, NOT, multiplexers, adders
 - No memory, no clock
- 2. **Sequential Logic** – Output depends on past inputs too (state)
 - Examples: Flip-flops, counters, shift registers 
 - Needs a clock signal to work
- 3. **Mixed Logic** – Most designs combine both



Verilog

➤ HARDWARE DESCRIPTION LANGUAGE

Verilog is used in two main ways:

1. **Design (RTL)** – Describing how hardware should function.
2. **Verification** – Creating testbenches to ensure the design works as expected."

RTL stands for Register Transfer Level.

At this level:

- **Registers hold state (flip-flops).**
- **Combinational Logic processes inputs and produces outputs.**
- **Data Transfer happens between registers via logic on each clock cycle.**



Design & Testbench

DESIGN USED FOR GENERATING HARDWARE, TESTBENCH IS USED FOR VERIFYING THE DESIGN

3: and

Design code:

```
module and_gate (  
    input a, b,  
    output y  
);  
    assign y = a & b;  
endmodule
```

Result:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Testbench code:

```
module tb_and_gate;  
    reg a, b;  
    wire y;  
  
    and_gate uut (.a(a), .b(b), .y(y));  
  
    initial begin  
        $display("A B | Y");  
        a=0; b=0; #10 $display("%b %b | %b", a, b, y);  
        a=0; b=1; #10 $display("%b %b | %b", a, b, y);  
        a=1; b=0; #10 $display("%b %b | %b", a, b, y);  
        a=1; b=1; #10 $display("%b %b | %b", a, b, y);  
        $finish;  
    end  
endmodule
```



RTL Thinking vs Software Thinking

HARDWARE THINKING IS DIFFERENT FROM SOFTWARE THINKING

- Software runs instructions sequentially in time — one after the other.
- Hardware executes everything in parallel.

When designing hardware, think about:

- **Parallelism** – Multiple blocks operate at the same time.
- **Timing** – Operations are synchronized to clock cycles.
- **Resources** – You're creating real flip-flops, gates, and multiplexers.

Also, hardware has timing constraints. You must think:

- **What happens every clock edge?**
- **Where does the data come from?**
- **Is there enough time for logic to settle before the next clock?**



A Simple Example – Designing a 2-bit Counter

HARDWARE THINKING IS DIFFERENT FROM SOFTWARE THINKING

- **Inputs:** clock, reset
- **Logic:** Increment current count by 1 each clock
- **State:** Store the current count in flip-flops

In RTL terms:

- On every rising clock edge, check if reset is high.
- If reset, load 0 into the counter.
- else, counter = counter + 1.



A Simple Example – Designing a 2-bit Counter

- **Inputs:** clock, reset
- **Logic:** Increment current count by 1 each clock
- **State:** Store the current count in flip-flops

In RTL terms:

- On every rising clock edge, check if reset is high.
- If reset, load 0 into the counter.
- else, counter = counter + 1.



```
module counter_2bit (  
    input  clk,  
    input  rst,  
    output [1:0] count  
);  
  
    always @(posedge clk) begin  
        if (rst == 1)  
            count <= 2'b00;  
        else  
            count <= count + 1;  
        end  
    endmodule
```

A Simple Example – Designing a 2-bit Counter



```
module counter_2bit (  
    input  clk,  
    input  rst,  
    output [1:0] count  
);  
  
    always @(posedge clk) begin  
        if (rst)  
            count <= 2'b00;  
        else  
            count <= count + 1;  
        end  
    end  
endmodule
```

```
module tb_counter_2bit;  
  
    reg clk, rst;  
    wire [1:0] count;  
  
    // DUT instance  
    counter_2bit uut (.clk(clk), .rst(rst), .count(count));  
  
    // Clock generation  
    initial clk = 0;  
    always #5 clk = ~clk; // 10ns period  
  
    initial begin  
        $display("Time\tRST\tCount");  
        $display("-----");  
  
        rst = 1; #10; // Apply reset  
        rst = 0; #50; // Run counter for a few cycles  
        rst = 1; #10; // Reset again  
        rst = 0; #30;  
  
        $finish;  
    end  
  
    initial begin  
        $monitor("%0t\t%b\t%b", $time, rst, count);  
    end  
endmodule
```

RTL Flow from Idea to Code

1. Understand the requirement – What is the function?
2. Draw the block diagram – What logic blocks and registers are needed?
3. Decide clocking – How many clocks? Synchronous or asynchronous reset?
4. Write RTL code – Use synthesizable Verilog constructs
5. Verify – Test with a simulation before synthesis

This is the exact process follow in industry when designing chips.

Why Design Matters for Verification

If you want to be a verification engineer, you must understand how designs are built.
Why? Because:

- You can predict where bugs are likely to occur
- You can write meaningful test cases
- You can debug waveforms faster
- You speak the same language as design engineers.



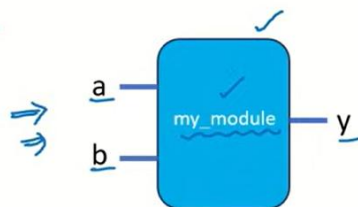
Structure of a Verilog Module

In Verilog, every design is described inside a module

A module has:

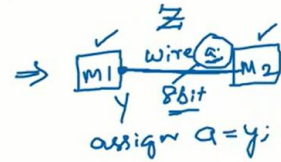
- **Name** – the identifier for your design.
- **Ports** – inputs and outputs for signals.
- **Body** – logic describing how outputs are generated from inputs.

and-gate



```
→ module my_module (  
  → input a, b,  
  → output y  
  );  
  // Design logic here  
→ endmodule
```

Data Types in Verilog



Verilog has several data types, but for RTL design we mainly use

- **Nets : wire** – Represents a physical connection between components. Must be driven by continuous assignments (assign) or outputs of other modules.

→ `wire signal_a; // one-bit value as a single net.`
 → `wire [5:0] signal_a; // net as a vector`

- * Default value of wire is Z

- **reg** – Stores a value until explicitly changed, used in procedural blocks (always or initial).

→ `reg reg_a; // single bit register`
 → `reg [7:0] reg_a; // 8 bit register as a vector`

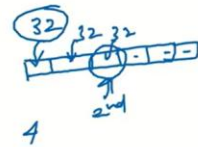
Always remember: index [MSB:LSB]

- * Default value of reg is x

- ✗ **logic** – System Verilog type that can replace both wire and reg in most cases.



Arrays and Memory



→ **Array** - Verilog allows arrays of reg, time, integer, and vector register data types.

→ `integer counts [0:5]; // array of 5 elements each can have integers (32 bit value)`

→ `count[2] // To access 2nd element in an array`

→ `integer two_d_arr [3:0][3:0]; // illegal declaration (verilog not allow multidimensional array)`

→ `reg [3:0] var [0:7]; // Array of 8 var and each var has 4 as bit width`

- Verilog does not allow the use of multidimensional arrays.
- Arrays for real variables are also not allowed in Verilog.

⇒ **Memory** - Verilog provides a facility to model register memories like ROM or RAM as an array of registers.
 Each array element in an array is called a word.

→ `reg mem [0:1023]; // Memory mem with 1024 1-bit word.`

→ `reg [8:0] mem [0:511]; // Memory mem with 512 8-bit words : 512 bytes`

→ `reg [8:0] mem [0:1023]; // memory of 1024 bytes = 1 KB`

Numbers in Verilog

Numbers in Verilog can be written with a specific format:

- > <size>'<base><value>
- Handwritten notes:*
- <size>: 4, 8, 16
- <base>: b, d, h, o
- <value>: 1010, 25, FF
- <size> = number of bits
 - <base> = b (binary), d (decimal), h (hex), o (octal)
 - <value> = the number itself
- > 4'b1010 // 4-bit binary 1010
> 8d25 // 8-bit decimal 25
> 16'hFF // 16-bit hex FF

If size is omitted, Verilog defaults to 32 bits."

Operators in Verilog

Operator Type	Operator symbols
⇒ Arithmetic Operators	+, -, *, /, %
⇒ Logical Operators	&&, , !
⇒ Bitwise Operators	&, , ~, ^
⇒ Relational Operators	==, !=, <, >, <=, >=, ===, !==
⇒ Shift Operators	<<, >>, <<<, >>>
⇒ Concatenation Operator	{}
⇒ Replication Operator	{n{}}

Arithmetic Operators

Operator	Meaning	Example	Result
+ ✓	Addition	5 + 3	8
- ✓	Subtraction	9 - 4	5
* ✓	Multiplication	3 * 4	12
/ ✓	Division (quotient)	9 / 4	2
% ✓	Modulus (remainder)	9 % 4	1

```

module tb_arithmetic;
    reg [3:0] a = 4'd9, b = 4'd4;
    wire [4:0] sum, diff, div, mod;
    wire [7:0] prod;

    assign sum = a + b; // 9 + 4 = 13
    assign diff = a - b; // 9 - 4 = 5
    assign prod = a * b; // 9 * 4 = 36
    assign div = a / b; // 9 / 4 = 2 (integer division)
    assign mod = a % b; // 9 % 4 = 1
endmodule

```

Logical Operators

Operator	Meaning	Example	Result
&&	Logical AND (1 if both true)	1 && 0	0
	Logical OR (1 if any one true)	1 0	1
!	Logical NOT	!1	0

```

module tb_logical;
    reg [3:0] x = 1100, y = 0000;
    wire and_result, or_result, not_result;

    assign and_result = x && y; // 1 && 0 = 0
    assign or_result = x || y; // 1 || 0 = 1
    assign not_result = !x; // !1 = 0
endmodule

```

logic gates

Bitwise Operators

Operator	Meaning	Example Code	Result
&	Bitwise AND	1010 & 1100	1000
	Bitwise OR	1010 1100	1110
^	Bitwise XOR	1010 ^ 1100	0110
~	Bitwise NOT	~1010	0101

```

module tb_bitwise;
    reg [3:0] p = 4'b1010, q = 4'b1100;
    wire [3:0] and_b, or_b, xor_b, not_b;

    assign and_b = p & q; // 1010 & 1100 = 1000
    assign or_b = p | q; // 1010 | 1100 = 1110
    assign xor_b = p ^ q; // 1010 ^ 1100 = 0110
    assign not_b = ~p; // ~1010 = 0101
endmodule

```

Relational Operators

Operator	Meaning	Example Code	Result
==	Equal to ✓	7 == 10	0 ✓
!=	Not equal to	7 != 10	1
<	Less than	7 < 10	1
>	Greater than	7 > 10	0
<=	Less than or equal	7 <= 10	1
>=	Greater than or equal	7 >= 10	0
===	Case equality (matches 0, 1, x, z)	2'b1x === 2'b1x	1
!==	Case inequality (differs in 0, 1, x, z)	2'b1x !== 2'b10	1

4 bit → 0, 1, x, z

```

module tb_relational;
    reg [3:0] a = 4'd7, b = 4'd10;
    wire eq, neq, lt, gt, lte, gte, ceq, cneq;
    reg [1:0] p = 2'b1x, q = 2'b1x;

    assign eq = (a == b); // 0 ✓
    assign neq = (a != b); // 1 ✓
    assign lt = (a < b); // 1 ✓
    assign gt = (a > b); // 0 ✓
    assign lte = (a <= b); // 1 ✓
    assign gte = (a >= b); // 1 ✓
    assign ceq = (p === q); // 1 ✓
    assign eq = (p == q); // x ✓
    assign cneq = (p !== q); // 0 ✓

    // Comment
    // 1x == 10 → 0
    // 1x === 10 → 0
endmodule

```

Shift Operators

Operator	Meaning	Example	Result
$\Rightarrow$ <u>$\ll$</u>	Shift left (fill with 0)	$1010 \ll 1$	0100
$\Rightarrow$ <u>$\gg$</u>	Shift right (fill with 0)	$1010 \gg 2$	0010
$\Rightarrow$ <u>$\lll$</u>	Arithmetic shift left (same as logical left for most cases)	$1100 \lll 2$	0000
$\Rightarrow$ <u>$\ggg$</u>	Arithmetic shift right (fills with sign bit)	$1100 \ggg 2$	1111

```

module tb_shift;
    reg [3:0] x = 4'b1010;
    wire [3:0] left, right, ar_right, ar_left;
    reg signed [3:0] s = -4; // binary 1100

    assign left = x << 1; // 1010 << 1 = 0100
    assign right = x >> 2; // 1010 >> 2 = 0010
    assign ar_left = s <<< 2; // 1100 <<< 2 = 0000 (Normal Left shift)
    assign ar_right = s >>> 2; // 1100 >>> 2 = 1111 (Keep MSB as it is)

endmodule

```

Concatenation Operator

Operator	Meaning	Example Code	Result
<u>$\{\}$</u>	Join multiple signals	<u>$\{2'b10, 2'b01\}$</u>	<u>1001</u>

$\{a, b, c\}$

```

module tb_concat;
    reg [1:0] a = 2'b10; ✓
    reg [1:0] b = 2'b01; ✓
    wire [3:0] c; ✓

    assign c = {a, b}; // combine 2 bits , 2 bits = 4 bits

    initial begin
        $display("a=%b, b=%b, {a,b}=%b", a, b, c); // c = 1001
    end
endmodule

```

Replication Operator

Operator	Meaning	Example Code	Result
<u>$\{n\}$</u>	Repeat a value n times	<u>$\{4\{1'b1\}\}$</u>	<u>1111</u>

```

module tb_replicate;
    reg x = 1'b1;
    wire [3:0] y;

    assign y = {4{x}}; // repeat bit '1' four times

    initial begin
        $display("x=%b, {4{x}}=%b", x, y); // y = 1111
    end
endmodule

```



Probable Interview / Test Questions

- Some Probable Interview / Test Questions

1. What are the differences between reg and wire in Verilog?
 2. How do you declare a 4-bit bus in Verilog?
 3. Explain the format <size>'<base><value> for representing numbers in Verilog with examples.
 4. What's the default bit-width in Verilog if size is not specified?
 5. Difference between logical operators (&&, ||, !) and bitwise operators (&, |, ^, ~).
 6. What are relational operators and where are they used in Verilog?
 7. Explain the difference between logical shift (<<, >>) and arithmetic shift (<<<, >>>).
 8. What is the use of the concatenation ({}) and replication ({n{}}) operators in Verilog?
 9. What is the difference between == and === in Verilog?
 10. Why does Verilog have case equality (===) and case inequality (!==) operators?
-



Day 3

we'll dive into **continuous assignments**, and **how to use wire, reg in real designs**.

- What is Continuous Assignment?
 - How to use Continuous Assignments?
 - Example designs
 - Conditional Operator
 - other net types and assignments
 - ✓ wire
 - ✓ tri (and variations: tri0, tri1)
 - ✓ wand (wired-AND)
 - ✓ wor (wired-OR)
 - ✓ supply0
 - ✓ supply1
-



What is Continuous Assignment?

In Verilog, **continuous assignment** means a signal is **continuously driven** with the value of an expression.

- ✓ Uses the **assign** keyword
- ✓ Works only with **wire**-type variables (**not reg**).
- ✓ **No clock, No always block** — pure logic

```
wire y;  
assign y = a & b;
```

```
assign <net_name> = <expression>;
```

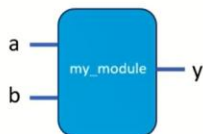
<net_name> must be a net type (wire, tri, etc.).

<expression> can be constants, signals, operators.



How to use Continuous Assignment?

In Verilog, **continuous assignment** means a signal is **continuously driven** with the value of an expression.

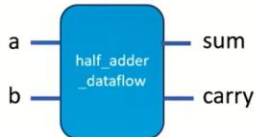


```
module my_module (  
    input  a, b,  
    output y  
);  
    // Design logic here  
endmodule
```

```
module my_module (  
    input wire a, b,  
    output wire y;  
);  
    assign y = a & b; // continuous assignment  
endmodule
```

+ Half Adder

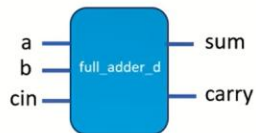
- ✓ A half adder is a fundamental digital circuit that performs the addition of two single-bit binary numbers.
- ✓ It has two inputs, let's call them 'a' and 'b', and two outputs: 'sum' and 'carry'.



```
module half_adder_dataflow (  
    input a, // First input bit  
    input b, // Second input bit  
    output sum, // Sum output  
    output carry // Carry output  
);  
    assign sum = a ^ b; // Sum is the XOR of a and b  
    assign carry = a & b; // Carry is the AND of a and b  
endmodule
```

+ Full Adder

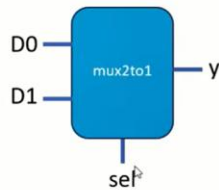
- ✓ A full adder is a fundamental digital circuit that performs the addition of three single-bit binary numbers.
- ✓ It has three inputs, let's call them 'a', 'b' and 'cin', and two outputs: 'sum' and 'carry'.



```
module full_adder_d (  
    input a, b, cin,  
    output sum, carry  
);  
    assign sum = a ^ b ^ cin;  
    assign carry = (a & b) | (b & cin) | (cin & a);  
endmodule
```


2-to-1 Multiplexer

- ✓ A 2-to-1 multiplexer, often abbreviated as a 2:1 MUX, is a basic combinational logic circuit that chooses one of two input signals and sends it to a single output.
- ✓ It is a fundamental component in digital electronics and acts like a data selector.



```
module mux2to1(input wire D0, D1, sel, output wire y);  
    assign Y = (~sel & D0) | (sel & D1);  
endmodule
```

Inputs and outputs

- Data Inputs: It has two data input lines, typically labeled D0 and D1.
- Select Line: It has a single control or select input, denoted as S.
- Output: It has one output line, labeled Y.

Conditional Operator

<condition> ? <true_expression> : <false_expression>;

<condition>: A logical expression evaluated to be true or false.

<true_expression>: The expression whose value is assigned if the condition is true (non-zero).

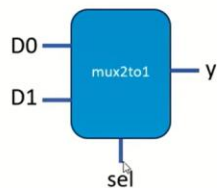
<false_expression>: The expression whose value is assigned if the condition is false (zero)

```
assign out = enable ? data1 : data2;
```

if enable is true (1), out is assigned the value of data1.
Otherwise, out is assigned the value of data2

Similar to if else

```
if(enable)  
    out = data1;  
else  
    out = data2;
```



```
module mux2to1(input wire D0, D1, sel, output wire y);  
    assign y = (sel) ? D1 : D0;  
endmodule
```



Nested Conditional Operator

```
<condition1> ? (<condition2> ? <true_expression1> : <false_expression1>) : (<condition3> ? <true_expression2> : <false_expression2>);
```

```
assign y = s[1] ? (s[0] ? d3 : d2) : (s[0] ? d1 : d0);
```

s[1]	s[0]	y
0	0	d0
0	1	d1
1	0	d2
1	1	d3

```
module mux4to1(input wire d0, d1, d2, d3, input [1:0]s, output wire y);  
    assign y = s[1] ? (s[0] ? d3 : d2) : (s[0] ? d1 : d0);  
endmodule
```



Types

- ✓ Explicit Continuous Assignment:
This requires a separate declaration of the net.
- ✓ then a subsequent assign statement to continuously assign a value to it.

```
wire out; // Explicit declaration  
assign out = (a == b) ? 1'b1 : 1'b0; // Explicit assignment
```

- ✓ Implicit Continuous Assignment:
This combines the net declaration and the continuous assignment into a single statement.

```
wire out = (a == b) ? 1'b1 : 1'b0; // Implicit declaration and assignment
```

✗ Common mistakes

- ✓ Using assign with reg — not allowed.
- ✓ Trying to use assign inside an initial, always block — wrong.
- ✓ continuous assignments are declared outside of these blocks
- ✓ Reg declared outputs are assigned using assign — wrong.

```
reg y;  
assign y = a & b; // ✗ Compilation error
```

```
reg a, b;  
wire y;  
assign y = a & b; // ✓ Continuous assignment
```

✓ Common Practices

- ✓ Continuous assignments are ideal for describing combinational logic circuits such as AND gates, OR gates, multiplexers, decoders, and other circuits where the output is solely a function of the current inputs.

Other net types and usage

1 wire

What it does: Basic net that connects hardware elements; continuously reflects driven value.

Where used: Point-to-point connections between combinational logic and module ports.

Synthesizable?  Yes — synthesizes to a direct wire connection or gate output.

```
reg a, b;  
wire y; // Simple net connection  
assign y = a & b; // Continuous assignment
```

2 tri

What it does: Net type for tri-state logic; can have multiple drivers, or be high-impedance (z) when not driven.

Where used: Shared buses (e.g., inout ports) where only one device drives at a time.

Synthesizable?  Yes — synthesizes to FPGA/ASIC tri-state buffers (FPGA may restrict to I/O pins).

```
reg d1, d2;  
reg en1, en2;  
tri bus; // Shared bus  
  
assign bus = (en1) ? d1 : 1'bz;  
assign bus = (en2) ? d2 : 1'bz;
```

Other net types and usage

tri0 – Default Low when not Driven

 **Usage:** Bus that defaults to 0 when idle.

```
reg en;  
tri0 y; // Defaults to 0 when no driver  
  
assign y = (en) ? 1'b1 : 1'bz;
```

tri1 – Default High when not Driven

 **Usage:** Bus that defaults to 1 when idle.

```
reg en;  
tri1 y; // Defaults to 1 when no driver  
  
assign y = (en) ? 1'b0 : 1'bz;
```

Other net types and usage

5 wand (wired-AND)

What it does: Multi-driver net where outputs are ANDed together automatically.

Where used: Open-collector / open-drain outputs in hardware, multiple active-low signal sources.

Synthesizable? ⚠ Rarely synthesizable — most synthesis tools require explicit AND logic instead.

```
module wand_example;
  reg a, b;
  wand y; // wired-AND net

  assign y = a; // driver 1
  assign y = b; // driver 2

  initial begin
    a = 1; b = 1; #5; $display("y=%b", y); // 1 & 1 = 1
    a = 1; b = 0; #5; $display("y=%b", y); // 1 & 0 = 0
    a = 0; b = 0; #5; $display("y=%b", y); // 0 & 0 = 0
  end
endmodule
```



💡 **Usage:** Multiple drivers ANDed together automatically.

Other net types and usage

6 wor (wired-OR)

What it does: Multi-driver net where outputs are ORed together automatically.

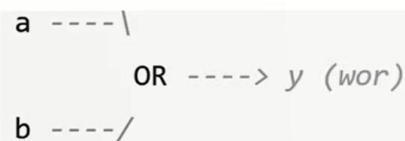
Where used: Open-source outputs, interrupt lines, or wired-OR buses.

Synthesizable? ⚠ Rarely synthesizable — may require explicit OR logic for synthesis.

```
module wor_example;
  reg a, b;
  wor y; // wired-OR net

  assign y = a; // driver 1
  assign y = b; // driver 2

  initial begin
    a = 0; b = 0; #5; $display("y=%b", y); // 0 | 0 = 0
    a = 1; b = 0; #5; $display("y=%b", y); // 1 | 0 = 1
    a = 0; b = 1; #5; $display("y=%b", y); // 0 | 1 = 1
  end
endmodule
```



💡 **Usage:** Multiple drivers ORed together automatically.

Other net types and usage

◆ supply0

What it does: Constantly drives logic 0 at supply strength (strongest possible).

Where used: Tying signals permanently to ground (GND).

Synthesizable? ☒ Yes — synthesizes to a direct connection to ground.

```
supply0 gnd;  
wire a;  
assign a = gnd; // a will always be 0
```

◆ supply1

What it does: Constantly drives logic 1 at supply strength.

Where used: Tying signals permanently to Vdd (logic high).

Synthesizable? ☒ Yes — synthesizes to a direct connection to Vdd.

```
supply1 vdd;  
wire b;  
assign b = vdd; // b will always be 1
```

What is Procedural Assignment?

- ✓ assign can only drive net types like wire — it's not used for storage or clocked logic.
- ✓ In Verilog, a **Procedural Assignment** assigns a value to a **variable** inside a procedural block.
 - ✓ ✓ initial block → happens only once at time 0.
 - ✓ ✓ always block → happens repeatedly based on events/triggers.

```
→ initial  
    ~ variable = value; ✓  
  
always @(sensitivity_list)  
    variable = expression; // or <= expression;
```
